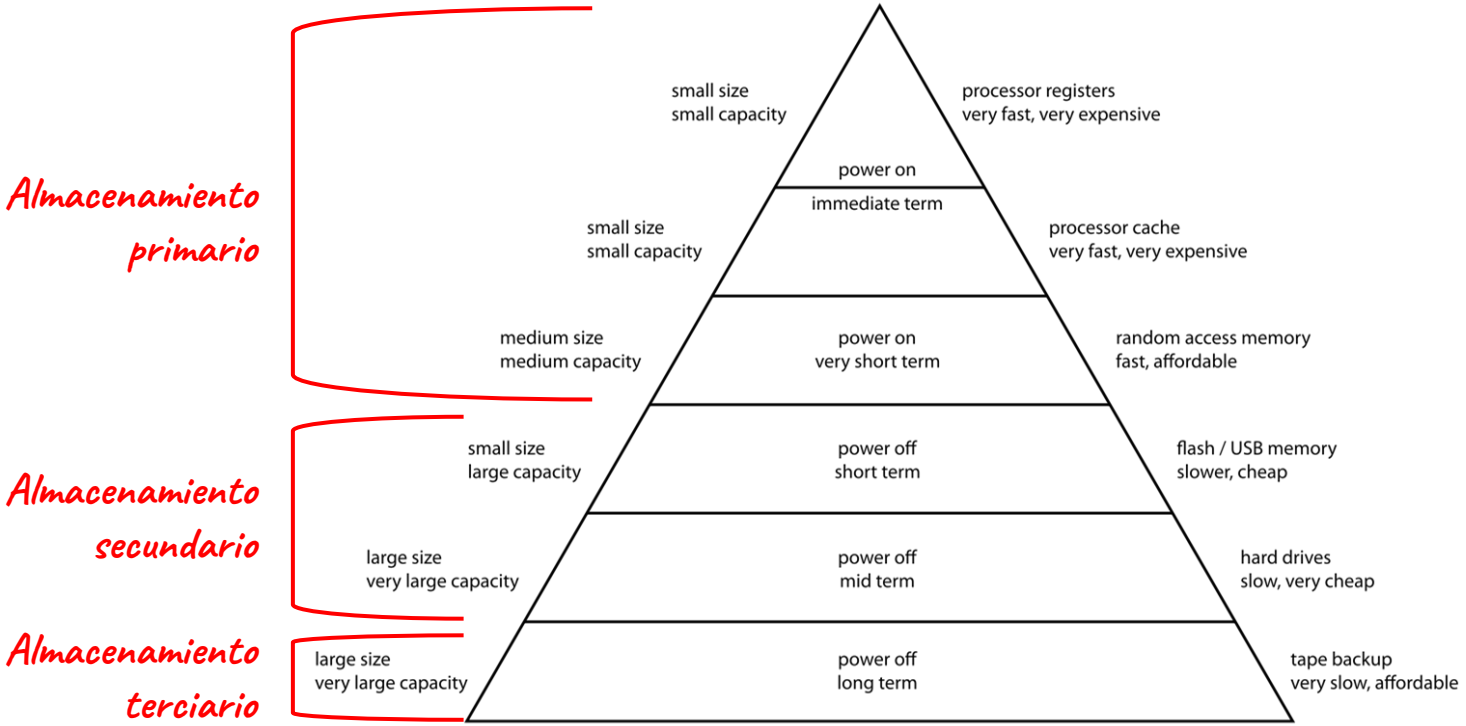
UNIVERSIDAD TORCUATO DI TELLA

TD7: Ingeniería de Datos

Almacenamiento e Índices

Almacenamiento

Tipos de almacenamiento



Discos magnéticos

- El almacenamiento en disco se organiza en **bloques**
  - Son de longitud fija (usualmente entre 4kb y 16kb)
  - Se manipulan de forma atómica (se leen y se escriben en su totalidad)
  - Leerlos implica copiar su contenido a la memoria principal
- El **acceso secuencial (sequential access)** implica leer bloques *físicamente* adyacentes, mientras que el **acceso aleatorio (random access)** se da cuando se solicitan bloques dispersos.

Discos magnéticos

- Técnicas de optimización
  - **Buffering:** permite almacenar los bloques ya leídos en memoria para acelerar futuras lecturas. Cuando se pide un bloque que ya está en el buffer, no es necesario acceder al disco.
  - **Read-ahead:** acelera las lecturas secuenciales almacenando bloques consecutivos en memoria incluso si todavía no se solicitaron. (Adelantarse)
  - **Scheduling:** organiza los pedidos de bloques para optimizar su acceso en el disco ordenándolos según su ubicación física.

Organización de archivos - Registros

- Los **registros** son el correlato en disco de las tuplas de cada relación.
- Organizan el almacenamiento de los **datos**.
- No deben superar el **tamaño del bloque**.
  - Y ningún registro debe atravesar las fronteras del bloque
- Pueden ser de tamaño **fijo o variable**.

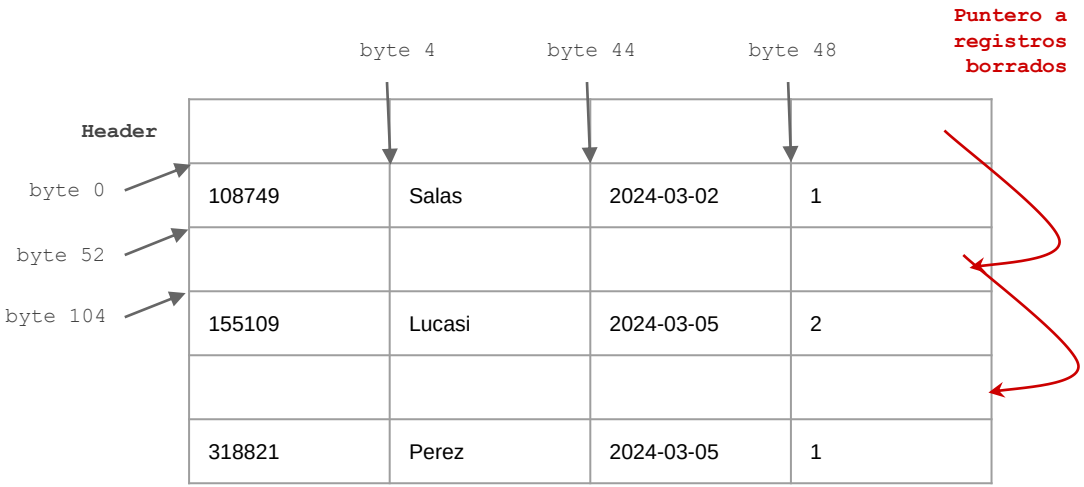
Organización de archivos - Registros

- La organización en **registros de tamaño fijo** facilita el almacenamiento, pero impide dominios de tamaño variable (como VARCHAR)
- Cada bloque tendrá tantos registros como *floor(tamaño de bloque / tamaño de registro)*
- Al borrar registros, va a ser necesario dejar “huecos” que luego se pueden volver a llenar en futuras inserciones

Organización de archivos - Registros

Fixed-length records

- legajo: 4 bytes
- apellido: 40 bytes
- fecha_alta: 4 bytes
- cant_hijos: 4 bytes



Organización de archivos - Registros

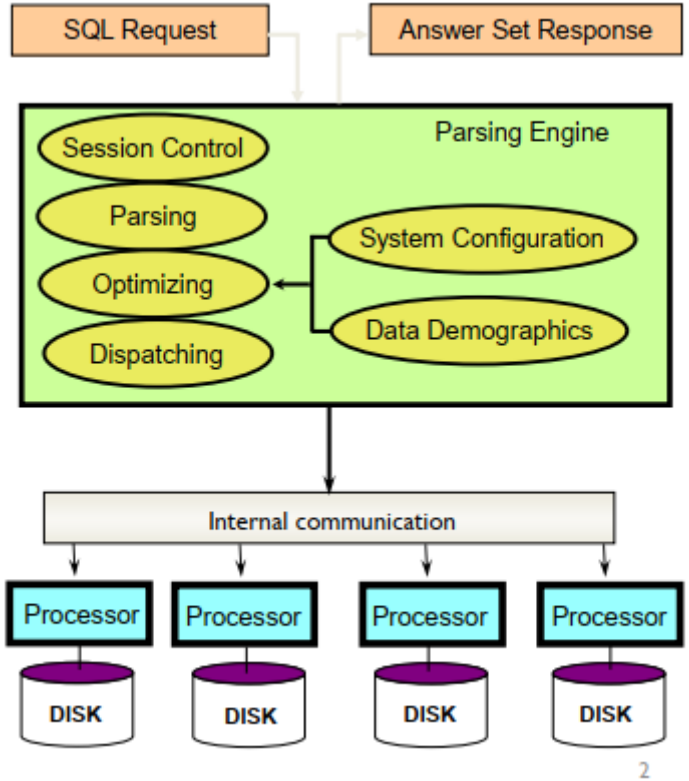
- La organización en **registros de tamaño variable** permite dominios de distinto tamaño sin necesidad de pre alocar espacio
- Permiten almacenar más de una relación en el mismo archivo
- Implican dificultades extra para poder encontrar los valores específicos de un atributo y la ubicación de cada registro
- Se utilizan headers a nivel bloque y a nivel registro

Organización de archivos - Registros

Variable-length records

- Cada bloque tendrá un header con información sobre sus registros:
  - Cantidad total de registros del bloque
  - Una lista (de tamaño fijo) con las ubicaciones de cada registro
  - Ubicación en el bloque del espacio vacío (reservado para futuros registros)

Almacenamiento - Metadata



DSC19, 13.4

Almacenamiento - Metadata

- Para poder operar con los datos almacenados, el motor también tiene que mantener **metadata** (**data dictionary**):
  - Nombres de relaciones, atributos y sus tipos
  - Constraints (como las PKs y FKs) e índices
  - Usuarios y permisos de acceso
  - Ubicación de las relaciones en el filesystem

Índices

Índices y optimización

```
SELECT Name, Composer
FROM Track
WHERE TrackId = 1752
```

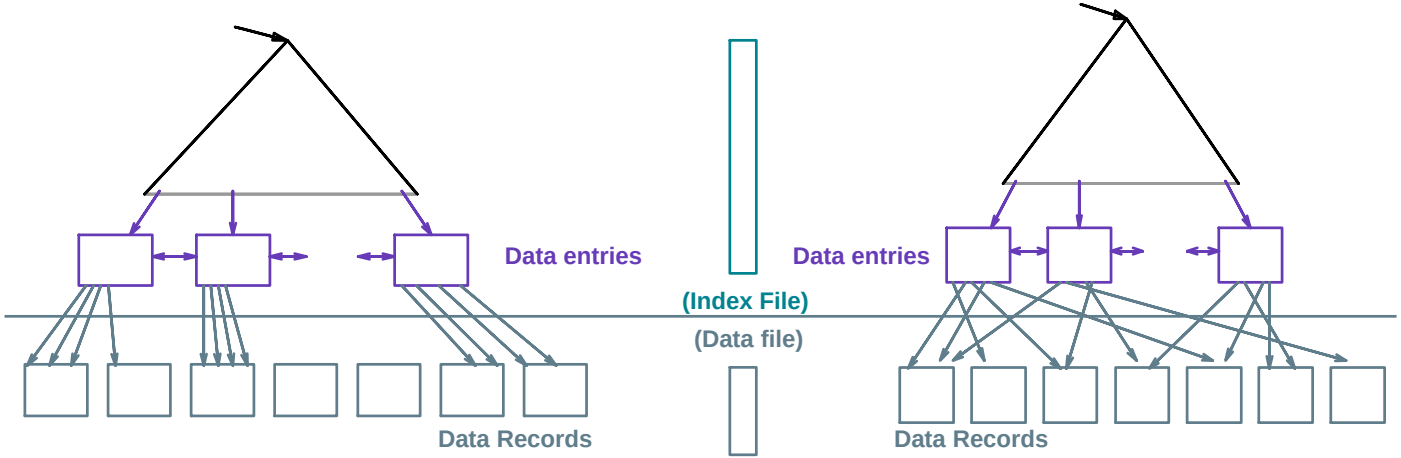
- ¿Cómo hace el motor de base de datos para resolver esta consulta?
- ¿Cómo accede a los registros y cómo elige con cuáles quedarse?

Índices y optimización

```
SELECT Name, Composer
FROM Track
WHERE TrackId = 1752
```

- En principio implica un **full scan**:
  - Se pide cada registro de la relación Track y se lo envía a memoria
  - Se evalúa su valor para el atributo TrackId
  - Si cumple con la condición, se agrega al resultado

Clustered vs. Unclustered Index



CLUSTERED

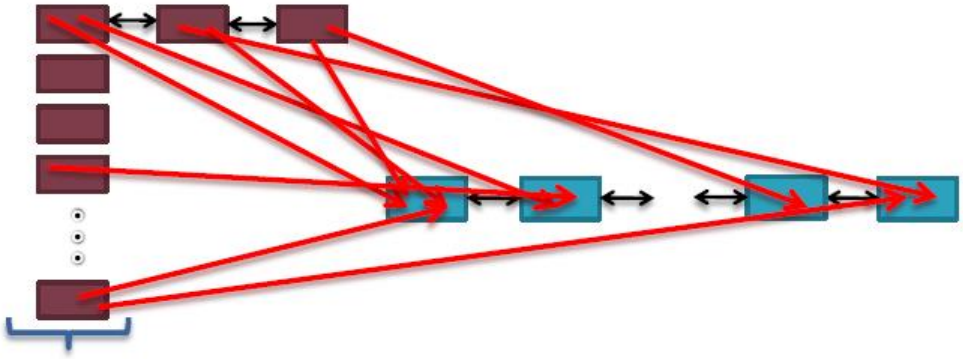
UNCLUSTERED

Índices Hash

Una tabla de hash almacena las claves de búsqueda

Cada posición de una tabla hash se asocia con un conjunto de registros. Por esta razón cada posición suele llamarse “un bucket”, y los valores de hash, “índices bucket”.

En cada bucket → **cantidad variable de bloques**



Chinook2/postgres@BD

Query Editor | Query History

```
1  -- Table: public.album
2
3  CREATE INDEX ifk_album_artist_id
4
5      ON public.album
6
7      USING btree
8
9      (artist_id ASC NULLS LAST)
10
11     TABLESPACE pg_default;
```

nombre del índice

para qué tabla se crea

tipo de índice

campo; orden asc; nulls al final

en qué tabla se crea

24th June 2021: PostgreSQL 14 Beta 2 Released!

This documentation is for an unsupported version of PostgreSQL.
You may want to view the same page for the **current** version, or one of the other supported versions listed above instead.

11.2. Index Types

PostgreSQL provides several index types: B-tree, Hash, GiST and GIN. Each index type uses a different algorithm that is best suited to different types of queries. By default, the `CREATE INDEX` command will create a B-tree index, which fits the most common situations.

B-trees can handle equality and range queries on data that can be sorted into some ordering. In particular, the PostgreSQL query planner will consider using a B-tree index whenever an indexed column is involved in a comparison using one of these operators:

```
<
<=
=
>=
>
```

Hash indexes can only handle simple equality comparisons. The query planner will consider using a hash index whenever an indexed column is involved in a comparison using the = operator.

Hash

Un hash es un mapeo de un conjunto a otro.

Ejemplo de función de hash:

tomar un número natural y hacer una suma iterativa de dígitos

hash (179) -> 17 -> 8

necesito 9 posiciones para mi índice

¿Dónde está el 179? -> buscar el índice 8, y recorrer todos los buckets

¿si quiero los números ordenados? Esta función de hash no ayuda.

179 278 197 ocupan buckets en el mismo índice

La única ventaja de Hash es la velocidad de acceso. Pero eso depende de las colisiones.

Índices

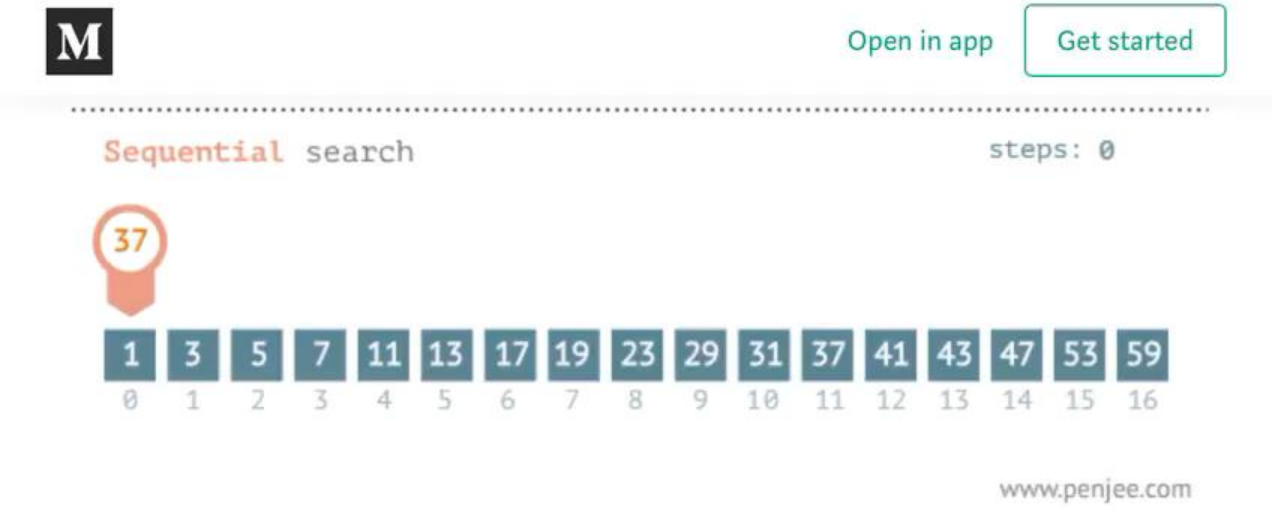
- Un **índice** es una estructura de datos para optimizar operaciones de búsqueda (y ordenamiento).
- Su uso correcto es fundamental para la performance de un sistema.
- Tiene dos contrapartidas:
  - Costo de espacio para **almacenar el índice** (disco y memoria)
  - Costo extra para mantener el índice actualizado ante cada **operación de escritura** que impacte en ese índice.

Índices

- Los índices se construyen sobre una **clave (search key)** que puede ser un atributo o un conjunto ordenado de atributos.
  - Permiten optimizar búsquedas por un atributo (por ejemplo *TrackId*)
  - Si están construidos sobre **múltiples atributos**, optimizan búsquedas o bien por el primer atributo, o bien por el primero + el segundo, o bien por el primero + el segundo + el tercero, etc…
- **Range queries**
  - Los índices optimizan tanto búsquedas por un valor como búsquedas de rango.

Índices

- Clustering indices (primary indices)
  - Definen el orden de los registros en el archivo
  - Pueden ser densos o ralos (apuntar sólo a algunos registros)
- Secondary indices
  - Son independientes del orden de los registros
  - Siempre densos: una entrada por cada valor existente de la search-key y punteros a cada registro de la relación
  - Unique y non-unique search-keys



Búsqueda lineal

Índices

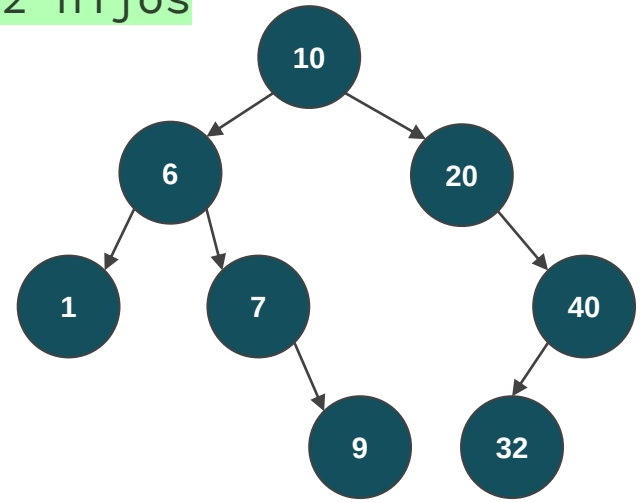
Según la estructura de datos sobre la que se construyen, pueden ser:

- Índices **ordenados**:
  - Se apoyan en estructuras de ordenamiento
  - Usualmente, la complejidad de lectura y escritura será *O(log n)*
- Índices de **hashes**:
  - Se basan en estructuras de hash table
  - Su acceso será *O(1)*, pero…
  - No servirán para búsquedas de rango

Árboles de búsqueda

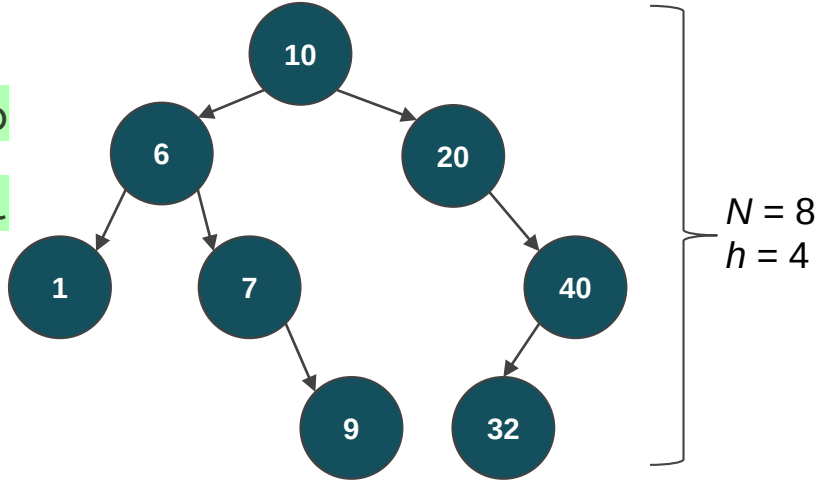
Índices ordenados - árboles de búsqueda

- Caso por antonomasia: **árbol de búsqueda binario**
- Cada nodo del árbol tiene como máximo 2 hijos
- Para cada nodo:
  - Todos los elementos del subárbol izquierdo son **menores** al valor del nodo
  - Todos los elementos del subárbol derecho son **mayores** al valor del nodo



Índices ordenados - árboles binarios

- Buscar un elemento implica una búsqueda binaria
- Una búsqueda será tan cara como la altura del árbol
  - Su complejidad será **log(n)** si el árbol está balanceado
  - En el peor caso de un árbol desbalanceado, será **n** O(n)



Índices ordenados - árboles AVL

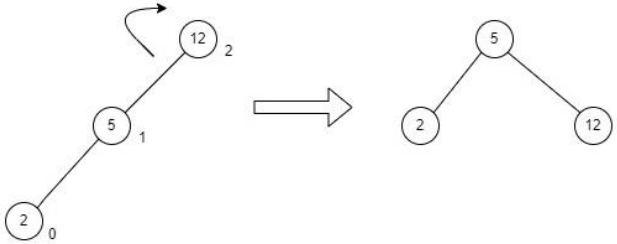
- El AVL es un árbol de búsqueda auto balanceante
- En un árbol AVL:
  - Todos los nodos contienen un valor único
  - La altura entre los dos subárboles hijos de un nodo puede diferir como máximo en 1
  - En cada inserción o borrado se puede producir un rebalanceo
  - Rebalancear implica una o más (como máximo H) operaciones de rotación (RR, LL, LR o RL) entre nodos

Índices ordenados - árboles AVL

- Después de insertar o eliminar un elemento, se recalcula la altura de los subárboles afectados (todo el path del nodo modificado)
- Al recorrer los nodos hacia la raíz, cuando se encuentra un desbalance (diferencia de altura mayor a 1) entre dos subárboles, se realiza una rotación:
  - **LeftLeft**: cuando el subárbol izquierdo está excedido por su lado izquierdo
  - **RightRight**: cuando el subárbol derecho está excedido por su lado derecho
  - **LeftRight**: cuando el subárbol izquierdo está excedido por su lado derecho
  - **RightLeft**: cuando el subárbol derecho está excedido por su lado izquierdo
- Una vez resuelto el desbalance local, se se continúa aplicando el mismo algoritmo hasta llegar a la raíz

Índices ordenados - árboles AVL

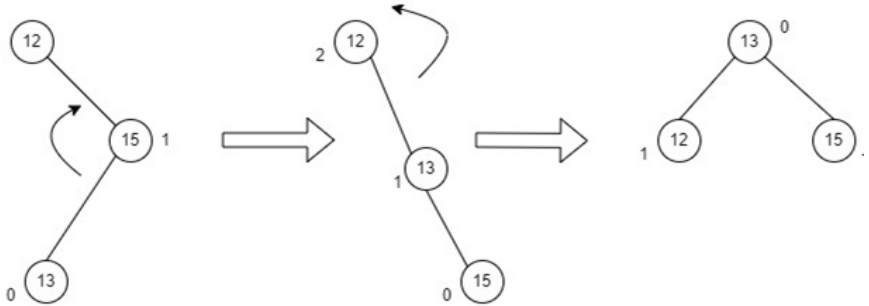
- LeftLeft rotation:



- El hijo izquierdo (5) y el nodo desbalanceado (12) se intercambian entre sí, **manteniendo cada uno sus hijos**.

Índices ordenados - árboles AVL

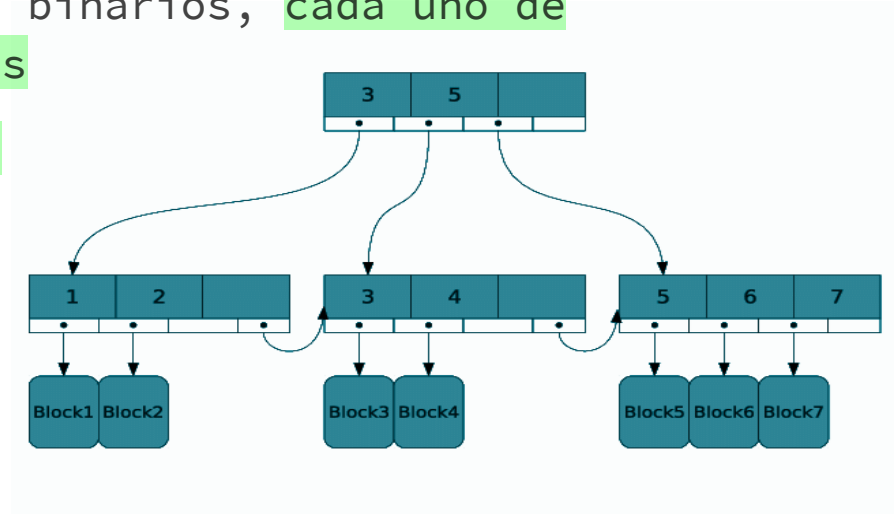
- RightLeft rotation:



- El hijo izquierdo (13) del hijo derecho (15) del nodo desbalanceado (12) se intercambia con su padre (15)
- Luego, el nuevo hijo derecho (13) y el nodo desbalanceado (12) se intercambian entre sí

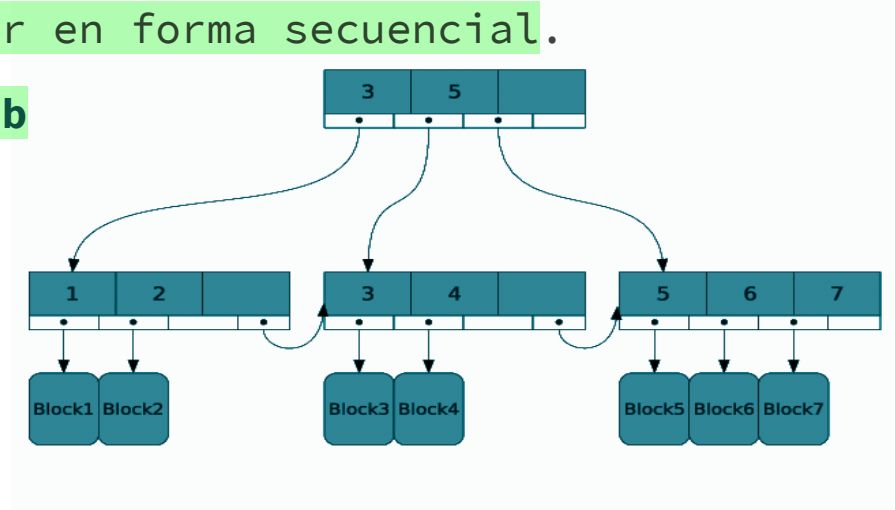
Índices - Árboles B+

- Los **árboles B+** son un tipo de árbol balanceado con alto nivel de branching
- En contraste con los árboles binarios, cada uno de los nodos tiene hasta **b** hijos
- Las hojas son las únicas que almacenan valores
- Las hojas están conectadas a modo de lista enlazada

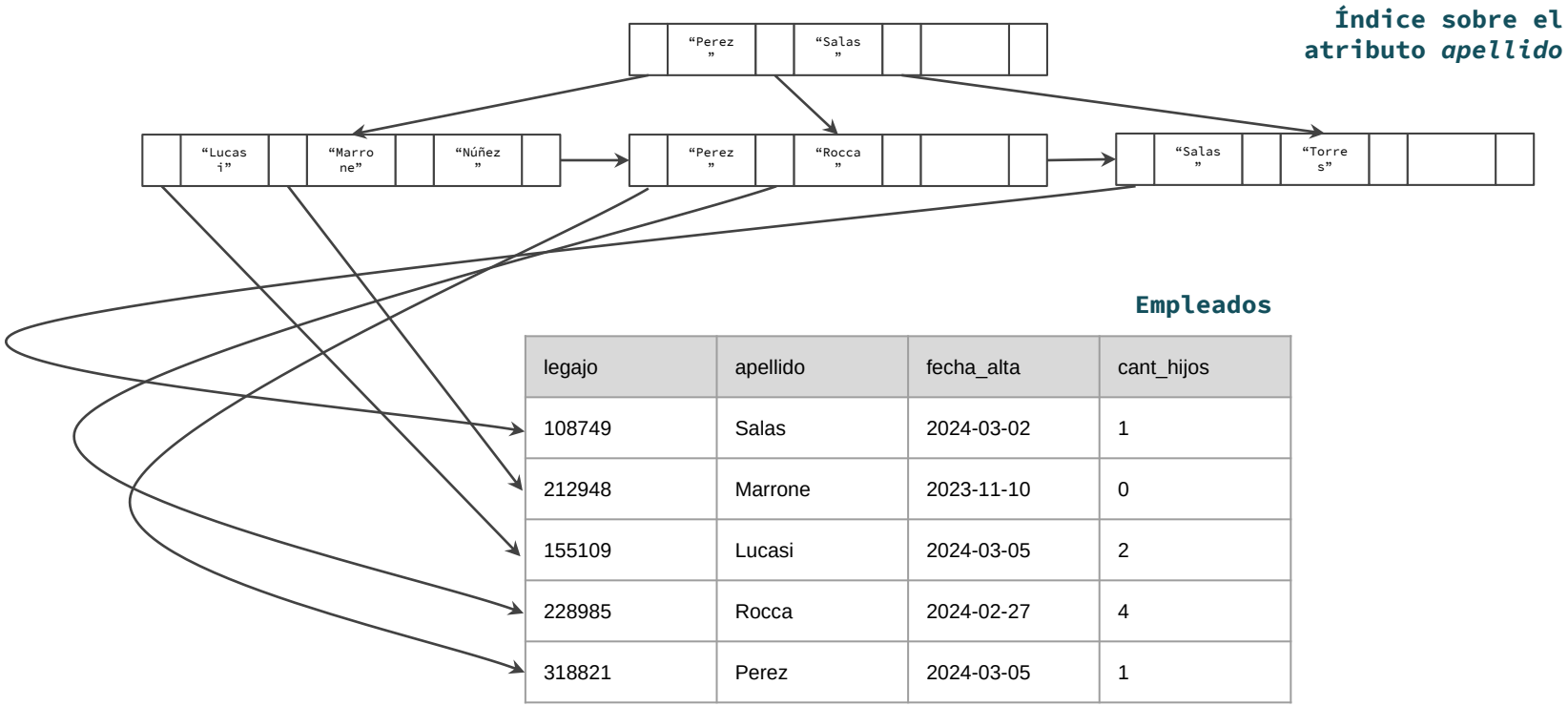


Índices - Árboles B+

- Todos los caminos de la raíz a una hoja tienen la misma altura
- Al tener alto branching, son de baja altura
- Permiten búsquedas y recorrer en forma secuencial.
- Cada nodo puede tener hasta **b** valores y **b+1** punteros.



Índices - Árboles B+



Índices - Árboles B+

- Cuando la **search-key** no es una clave candidata, se almacenan los valores repetidos en las hojas
- Para una **search-key** compuesta, los valores del índice también son compuestos.
  - Por ejemplo, para la search-key *[apellido, fecha_alta]*:

	“Lucasi”, 2024-03-05		“Marrone”, 2023-11-10		“Núñez”, 2021-04-08	

  - ¿Para qué búsquedas nos sirve este índice?

Índices - Árboles B+

- Generalmente se va a buscar que el tamaño del nodo sea equivalente al de un bloque:
  - De esta forma cada nodo se carga en memoria con **un** random access
  - Habrá tantas **operaciones de I/O** como la altura del árbol

- La búsqueda (de un valor puntual), el insert y el delete serán $O(log_b\ n)$

SQL

Índices en SQL

Para crear un índice en SQL tenemos que establecer su nombre, a qué tabla corresponde y qué atributo o atributos:

```
CREATE INDEX [nombre] ON [tabla]([lista de atributos]);
```

Por ejemplo:

```
CREATE INDEX apellido_idx ON empleados(apellido);
```

Índices en SQL

Para eliminarlo, usamos la sentencia DROP INDEX:

```
DROP INDEX [nombre];
```

En nuestro caso:

```
DROP INDEX apellido_idx;
```

Índices en SQL

- También se pueden crear índices **UNIQUE**
- Cumplen doble función:
  - Funcionar como índice de búsqueda
  - Refuerzan la unicidad de los valores de los atributos en cuestión

```
CREATE UNIQUE INDEX legajo_unq_idx
ON empleados(legajo);
```

- Toda **PK** tendrá automáticamente un **UNIQUE INDEX** creado
  - ¿Por qué es necesario un índice para garantizar unicidad?

Procesamiento de consultas

- El motor de bases de datos usa automáticamente índices cuando eso mejora la performance de una consulta
- En consultas complicadas, tiene que elegir entre distintos posibles planes de acción:

```
SELECT * FROM empleados WHERE cant_hijos < 2 AND
fecha_alta > '2021-01-01'
```

Cierre

Resumen

- El **almacenamiento** de los datos de una base se apoya en el concepto de **bloques**. Optimizar las formas de acceso para reducir los **accesos aleatorios** es fundamental.
- Las relaciones se almacenan organizando su información en **registros**.
- Los **índices** son estructuras de búsqueda que optimizan las consultas
  - Se basan principalmente en las estructuras de árboles de búsqueda balanceados
  - Optimizan búsquedas por valores puntuales o rangos y son necesarios para garantizar unicidad de atributos (como las PKs)